

ichosynth — Manuale di Costruzione

Versione DIY a 3 encoder, cablata a mano (senza PCB stampato)

Guida passo-passo per principianti: costruisci il tuo **ichosynth** con soli fili volanti (jumper), seguendo le tabelle dei pin.

ichosynth è il fork di **NI404** (di SP_ / soundpauli): un campionatore-sequencer open-source basato su **Teensy 4.1**. Genera tutti i suoni da solo, il computer serve **solo** per programmarlo la prima volta.

FORK

Questa è la build a 3 encoder. Monti **3 manopole** (SINISTRA, CENTRALE, DESTRA): più semplice ed economica dell'originale a 4. Il firmware del fork è già impostato per 3 encoder (`HAS_ENCODER4 0`): volume sulla SINISTRA, BPM sulla CENTRALE, e i comandi del 4° encoder rimappati sui 3 pulsanti. Altre aggiunte del fork: **OLED** di stato e **MIDI clock OUT** (entrambi opzionali).

Indice

- [1 · Cosa stai costruendo](#)
- [2 · Livello di difficoltà](#)
- [3 · Lista componenti \(BOM\)](#)
- [4 · Strumenti necessari](#)
- [5 · Sicurezza](#)
- [6 · Mappa pin completa](#)
- [7 · Montaggio passo-passo](#)
- [8 · Software: caricare il firmware](#)
- [9 · Preparare la micro SD](#)
- [10 · Primo avvio e test](#)
- [11 · Risoluzione problemi](#)
- [12 · Cheat-sheet pin](#)

1 · Cosa stai costruendo

- Un cervello: **Teensy 4.1** (un microcontrollore potente).
- Una scheda audio (**Teensy Audio Adaptor**) con uscita cuffie da 3,5 mm.
- Un display di gioco: una **matrice LED RGB 16×16** (256 LED).
- **Tre manopole rotative con pulsante** (encoder KY-040): SINISTRA, CENTRALE, DESTRA.
- Una **micro SD** dove stanno i campioni audio e i tuoi pattern.
- (*opzionale, aggiunta di questo fork*) un piccolo **schermo OLED** che mostra modalità, BPM, volume, ecc.

Tutto viene alimentato dalla porta **USB (5V)**.

2 · Livello di difficoltà — leggi prima di comprare

ATTENZIONE

Onestà prima di tutto. Il cablaggio è facile; la parte difficile è **una sola**: saldare i due chip PSRAM SMD sul retro del Teensy.

Parte	Difficoltà	Note
Cablaggio matrice + encoder	Facile	saldature grosse e collegamenti a filo, adatto a principianti pazienti
Saldatura PSRAM (2× chip SMD)	Difficile	componenti minuscoli; vedi opzioni sotto

- **PSRAM — opzione A (consigliata):** compra il Teensy 4.1 **con la PSRAM già saldata**, o fattela saldare da chi ha esperienza/stazione ad aria calda.
- **PSRAM — opzione B:** esercitati prima su schede di pratica.
- La PSRAM è **obbligatoria**: il firmware usa ~16 MB di memoria esterna (servono **due** chip da 8 MB). Senza, non parte.

Tempo stimato: mezza giornata per chi sa già saldare; di più se è la prima volta.

3 · Lista componenti (BOM)

Q.tà	Componente	Note
1	Teensy 4.1	il microcontrollore principale
2	Chip PSRAM 8 MB (APS6404, compatibile Teensy 4.1)	totale 16 MB, obbligatori
1	Teensy Audio Adaptor Board, Rev D (per Teensy 4.x)	codec SGTL5000 + jack 3,5 mm + slot SD (non usato)
1	Matrice LED WS2812B 16×16 (256 LED)	rigida o flessibile
3	Encoder rotativo KY-040 con pulsante	SINISTRA, CENTRALE, DESTRA (build a 3 encoder)
1	(<i>fork</i>) Pulsante momentaneo (tactile/push)	filtro lowpass — tra pin 41 e GND (vedi 6.5)
1	Micro SD Card, Class 10 , <= 32 GB	formattata FAT32
1	Cavo micro-USB + alimentatore 5V (>= 2A consigliato)	alimentazione e programmazione
1	Cuffie con jack 3,5 mm	ichosynth non ha altoparlanti
q.b.	Cavetti jumper Dupont (~10 cm), strip di pin header	per i collegamenti
1	(<i>opzionale, fork</i>) OLED SSD1306 0,96" 128×64 I2C	schermo informazioni
1	(<i>opzionale</i>) Contenitore stampato 3D	file STL in _DOCS/_ENCLOSURE/

NOTA

Licenze campioni: il progetto **non** include file audio. Userai i tuoi campioni (vedi [cap. 9](#)).

4 · Strumenti necessari

- Saldatore a punta fine + stagno (e fluxante, molto utile per la PSRAM).
- Tronchesino, spelafili, pinzette.
- "Terza mano" o morsa per tenere fermi i pezzi.
- Multimetro (continuità e cortocircuiti — **fondamentale**).
- Alcol isopropilico per pulire i residui di fluxante.

- Precauzioni antistatiche (braccialetto ESD): Teensy e PSRAM sono sensibili.
- (solo se saldi tu la PSRAM) stazione ad aria calda o saldatore di precisione.

5 · Concetti base di sicurezza

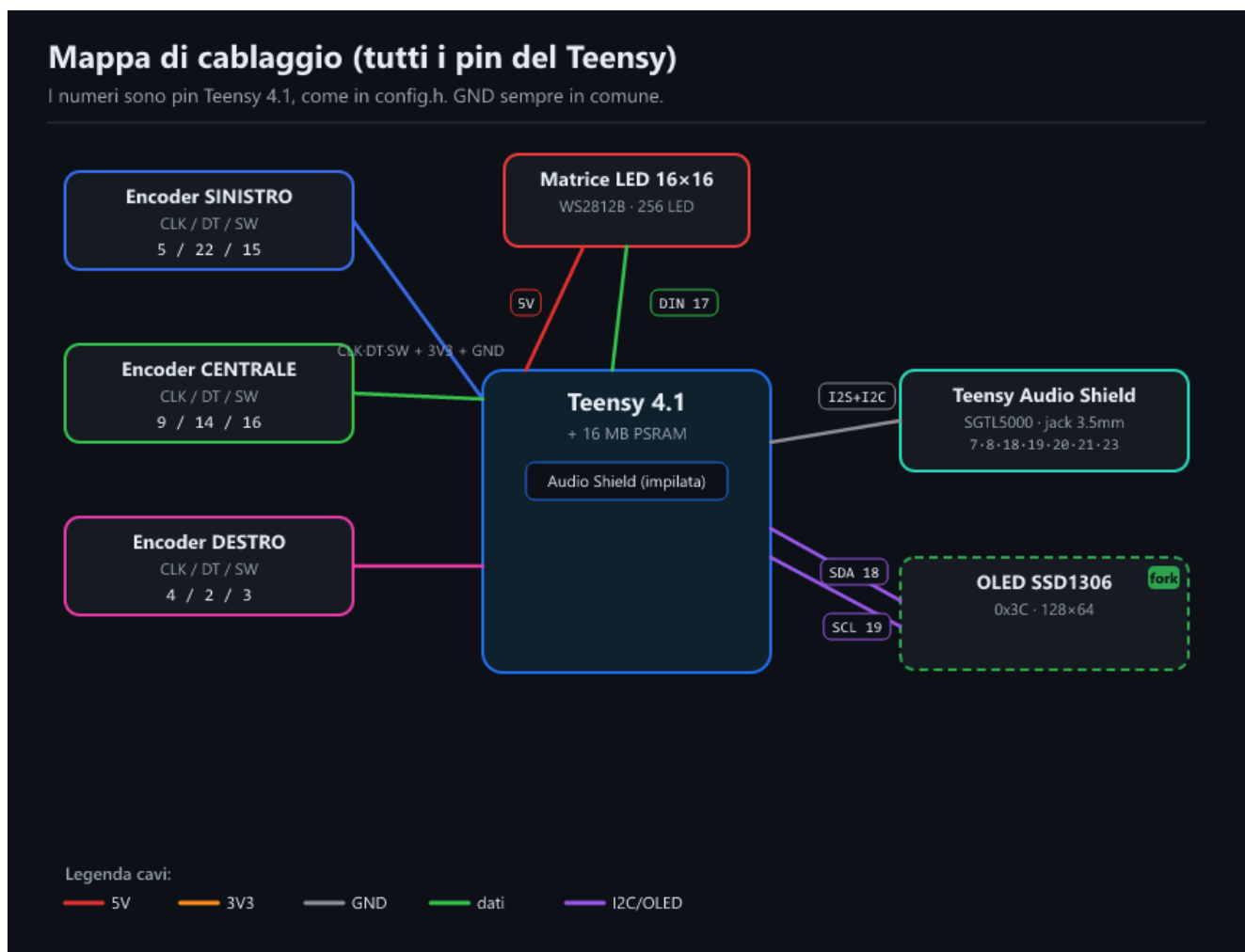
ATTENZIONE

Quattro regole che salvano i componenti (e i nervi):

1. **Mai** collegare/scollegare fili mentre il dispositivo è alimentato.
2. Doppio controllo **GND e 5V/3,3V** prima di dare corrente: invertirli può bruciare i componenti.
3. Dopo ogni fascio di saldature, col multimetro in continuità verifica che **non** ci siano corti tra 5V e GND.
4. Lavora con calma: una saldatura "fredda" (opaca, a pallina) è la causa #1 di malfunzionamenti.

6 · Mappa pin completa (la "verità" del firmware)

Questi pin sono definiti in [config.h](#) e sono ciò che il software si aspetta. **Wira esattamente questi numeri** (tutti pin del Teensy).



6.1 Matrice LED

Segnale matrice	Pin Teensy
DIN (dati)	17
+5V	5V
GND	GND

6.2 Scheda audio (Teensy Audio Adaptor, Rev D)

TIP

Il modo più semplice e affidabile è **impilare** la scheda audio sopra il Teensy con i pin header: così questi collegamenti si fanno da soli.

Se invece la cabli a mano, collega:

Segnale audio	Pin Teensy		Segnale audio	Pin Teensy
MCLK	23		SDA (I2C)	18
BCLK	21		SCL (I2C)	19
LRCLK (WS)	20		3,3V	3.3V
TX (DIN al codec)	7		GND	GND
RX (DOUT dal codec)	8			

NOTA

La SD si usa dallo **slot integrato del Teensy 4.1**, non da quello della scheda audio.

6.3 Encoder (CLK, DT, SW = pulsante)

Monti **3 encoder**. Ogni encoder ha 3 segnali (CLK, DT, SW) + alimentazione. Disponili da sinistra a destra come in tabella.

Encoder (ruolo nella build a 3 encoder)	CLK	DT	SW
SINISTRA — cursore su/giù, cancella, single · <i>volume</i>	5	22	15
CENTRALE — pagina, disegna nota, Play/Pausa, indietro · <i>BPM</i>	9	14	16
DESTRA — cursore sin/destra, mute, velocity	4	2	3

Inoltre, per ogni encoder: il pin "+" va a **3,3V**, il pin GND va a **GND**.

FORK

Il 4° encoder non si monta. Nel firmware è già impostato `#define HAS_ENCODER4 0`; i pin di rotazione del 4° encoder (CLK 32 / DT 33) sono a 99 = non usati. I comandi che l'originale metteva sul 4° encoder sono rimappati sui 3 pulsanti (vedi manuale d'uso, cap. 16). Il **seek** resta disattivato; il **filtro lowpass** è invece disponibile tramite un piccolo pulsante (vedi 6.5).

6.4 OLED opzionale (fork)

Condivide lo **stesso bus I2C dell'audio** (nessun pin extra: stesso SDA/SCL).

Segnale OLED	Pin Teensy
SDA	18

Segnale OLED	Pin Teensy
SCL	19
VCC	3,3V
GND	GND

NOTA

Indirizzo I2C predefinito **0x3C** (alcuni pannelli usano 0x3D).

6.5 Pulsante FILTRO (fork)

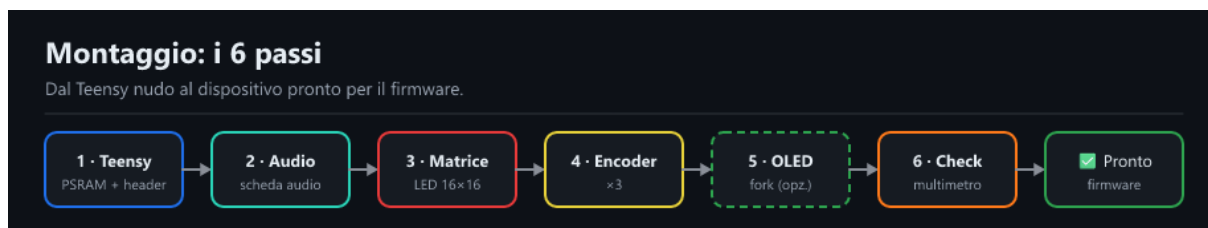
Un **comune pulsante momentaneo** (tipo tactile/push), che abilita il **filtro lowpass per voce** (vedi manuale d'uso, cap. 15). Non è un encoder: due soli fili.

Segnale pulsante	Pin Teensy
un capo	41
altro capo	GND

NOTA

Si usa la resistenza di **pull-up interna** del Teensy (pin attivo-basso): nessuna resistenza esterna. Il pin **41** è esattamente dove starebbe il pulsante del 4° encoder dell'originale. Non vuoi il filtro? Imposta `#define FILTER_ENABLED 0` in `config.h` e ometti il pulsante.

7 · Montaggio passo-passo



Fase 1 — Preparare il Teensy 4.1

1. Salda i **due chip PSRAM** nelle piazzole sul retro (vedi [cap. 2](#): se non te la senti, prendi un Teensy con PSRAM già montata).
2. Salda gli **header** sui bordi del Teensy (e quelli per la scheda audio se la impili).
3. Collega via USB al PC e verifica che venga riconosciuto (test completo in Fase 7/cap. 10).

Fase 2 — Scheda audio

1. Impila la scheda audio sul Teensy (consigliato) **oppure** cabla i segnali della [tabella 6.2](#).
2. Collega le cuffie al jack 3,5 mm (per i test).

Fase 3 — Matrice LED 16×16

1. Individua la **freccia di ingresso dati** (input): va al pin **17**. L'uscita (output) resta libera.
2. Collega **5V** e **GND** della matrice.
3. **Alimentazione**: il firmware usa colori a bassa luminosità, quindi di solito l'USB basta. Se in futuro alzi la luminosità, inietta i 5V da un alimentatore dedicato e **unisci le masse (GND comune)** tra Teensy e

matrice.

Fase 4 — Encoder (×3)

1. Per ciascuno dei **3 encoder** collega CLK, DT, SW secondo la [tabella 6.3](#), più "+" (3,3V) e GND.
2. Disponili da sinistra a destra: **SINISTRA -> CENTRALE -> DESTRA**.
3. Tieni i fili ordinati ed etichettali: incrociare CLK/DT è l'errore più comune (si corregge anche via software, vedi troubleshooting).
4. I pin di rotazione del **4° encoder** (32/33) restano **liberi**. (*Il pin 41 ospita il pulsante FILTRO opzionale — vedi 6.5.*)

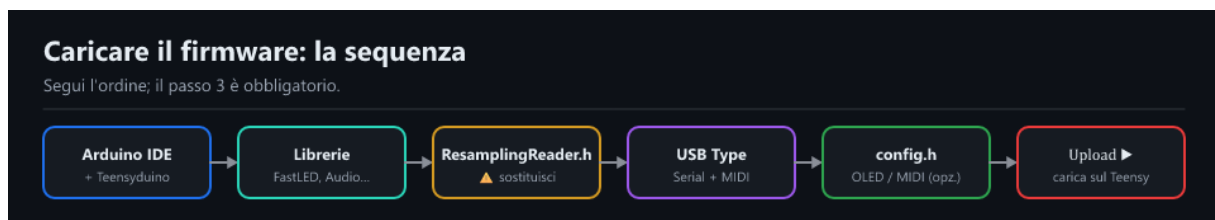
Fase 5 — OLED (opzionale)

1. Collega i 4 fili della [tabella 6.4](#). Essendo sullo stesso bus dell'audio, basta collegarsi in parallelo a SDA/SCL.

Fase 6 — Controllo finale prima di accendere

1. Col multimetro verifica **assenza di corto** tra 5V e GND e tra 3,3V e GND.
2. Ricontrolla che 5V/3,3V/GND siano nei pin giusti.

8 · Software: caricare il firmware



8.1 Installazione ambiente

1. Installa **Arduino IDE**.
2. Installa **Teensyduino** (l'add-on PJRC che aggiunge il supporto Teensy).

8.2 Librerie richieste

- WS2812Serial
- FastLED (>= 3.9.x)
- Encoder (Paul Stoffregen) — inclusa in Teensyduino
- Audio (Teensy Audio Library) — inclusa in Teensyduino
- Mapf
- TeensyPolyphony (newdigate)
- teensy-variable-playback (newdigate)
- avdweb_Switch
- **solo se usi l'OLED del fork:** Adafruit_SSD1306 e Adafruit_GFX

8.3 Passo OBBLIGATORIO: ResamplingReader.h

ATTENZIONE

Dentro la libreria teensy-variable-playback di newdigate, **sostituisci** il file ResamplingReader.h con quello fornito qui: [DOCS/ResamplingReader.h](#). Aiuta a evitare errori di puntatore nullo (nullptr).

8.4 Impostazioni di compilazione

Nel menu **Tools** di Arduino IDE:

- **Board:** Teensy 4.1
- **USB Type:** **Serial + MIDI** (*necessario: il MIDI passa dalla porta USB*)
- **CPU Speed:** predefinita (600 MHz)

8.5 (Fork) config.h: 3 encoder, OLED e MIDI clock out

In [config.h](#) la build a 3 encoder è **già impostata**:

```
#define HAS_ENCODER4 0           // build a 3 encoder (questa è la nostra)
```

Se vuoi, attiva anche le funzioni opzionali del fork mettendole a 1:

```
#define OLED_ENABLED 1           // attiva lo schermo OLED
#define MIDI_CLOCK_OUT_ENABLED 1 // invia il clock MIDI a strumenti esterni
```

FORK

OLED_ENABLED e MIDI_CLOCK_OUT_ENABLED a 0 (default) = nessuna funzione extra. HAS_ENCODER4 0 è la nostra build a 3 manopole; lascialo così.

8.6 Compilare e caricare

1. Apri `soundpauli_ni404.ino`.
2. Premi **Upload** (il Teensy entra in programmazione da solo; in caso premi il pulsantino sul Teensy).

9 · Preparare la micro SD (campioni)

Il firmware cerca i campioni sulla SD con questa **struttura precisa**:

Struttura della micro SD

Dove ichosynth cerca campioni, sample-pack e song.



`/samples/<cartella>/_<numero>.wav` con `<numero> = cartella*100 + indice`

Esempi reali (vedi cartella `_SDCARD/` del progetto):

```
/samples/0/_1.wav      (cartella 0, campione 1)
/samples/0/_99.wav     (cartella 0, campione 99)
/samples/1/_100.wav    (cartella 1, campione 0)
/samples/2/_200.wav    (cartella 2, campione 0)
```

Regole:

- Crea sulla **radice** della SD una cartella `samples`, e dentro le cartelle numerate `0`, `1`, `2`, ... (fino a `9`).
- I file devono chiamarsi `_<numero>.wav` con la numerazione qui sopra.
- **Formato audio richiesto: WAV mono, 16 bit, 44100 Hz.**

Convertire i tuoi campioni

Nella cartella `_SDCARD/` trovi gli strumenti che convertono qualsiasi WAV nel formato giusto e lo rinominano `_N.wav`.

`wavmaker.exe` — **interfaccia grafica (Windows, nessun Python richiesto), consigliato**

Doppio click: si apre una finestra. Poi:

1. **Aggiungi file** (o **Aggiungi cartella**) con i tuoi WAV — la lista mostra il **formato attuale** di ognuno e il **nome di destinazione** (`_1.wav`, `_2.wav`, ...). Le righe **verdi** sono già nel formato giusto.
2. Imposta il **Numero di partenza** (es. `1` per `samples/0`, `100` per `samples/1`, ...): la finestra ti ricorda in quale cartella SD finiranno.
3. Scegli la **Cartella di destinazione** (di default una nuova cartella `wav_convertiti`: gli

originali **NON** vengono toccati).

4. Premi **Converti**. Barra di avanzamento + log, poi sposti i `_N.wav` in `samples/<n>/` sulla SD.

La GUI è **non distruttiva**: scrive i convertiti in una cartella separata. Solo se spunti *"Elimina gli originali"* (con conferma) cancella i file di partenza.

Versioni Python (Windows/macOS/Linux, serve Python): `python wavmaker_gui.py` (stessa GUI) oppure `python wavmaker.py` (versione da riga di comando). Su Python 3.13+ serve `pip install audioop-lts`.

FILE

Sample pack (vedi manuale d'uso): cartelle numerate 1..99 sulla radice, ognuna con dentro `1.wav..12.wav`. Le crea/usa direttamente ichosynth dal menu, non devi prepararle a mano.

10 · Primo avvio e test

1. Inserisci la SD, collega le cuffie, alimenta via USB.
2. All'accensione vedrai una **animazione/logo** sulla matrice.
3. Se manca la SD compare l'icona **"noSD"** (rossa): spegni, inserisci la SD, riaccendi.
4. Muovi l'encoder **SINISTRA** e **DESTRA**: deve muoversi un puntino bianco pulsante (il cursore).
5. Premi (push) l'encoder **CENTRALE** per piazzare una nota: dovresti sentire il campione.
6. **Doppio click** sull'encoder **CENTRALE**: Play/Pausa.

Per imparare a suonarlo, vai al [Manuale d'Uso](#).

11 · Risoluzione problemi

Sintomo	Probabile causa / rimedio
Non si accende / non riconosciuto dal PC	Cavo USB solo-carica (usane uno dati); saldature header; corto 5V-GND
Si riavvia da solo / crash dopo pochi secondi	PSRAM mancante o mal saldata; alimentazione USB debole (usa 5V >= 2A)
LED non si accendono / colori sbagliati	DIN non sul pin 17; freccia dati invertita (usa l'ingresso); GND non comune
Solo alcuni LED accesi a caso	Alimentazione insufficiente alla matrice; GND comune mancante
Un encoder gira "al contrario"	Inverti i fili CLK e DT di quell'encoder
Un encoder non fa nulla	Pulsante/segnali sui pin sbagliati; ricontrolla tabella 6.3
Non riesco a fare Play	nella build a 3 encoder è doppio click sulla CENTRALE (vedi manuale d'uso, cap. 16)
Il filtro non cambia il suono	controlla il pulsante tra pin 41 e GND ; il filtro agisce sulla voce sotto il cursore (cap. 6.5)
Nessun suono in cuffia	Scheda audio non collegata bene (7,8,20,21,23,18,19); USB Type non impostato; volume a 0
Compilazione: errori nullptr / ResamplingReader	Non hai sostituito <code>ResamplingReader.h</code> (vedi 8.3)
Campioni non partono / canale muto	Struttura/percorso SD errati; WAV non mono-16bit-44.1k (usa <code>wavmaker.py</code>)
OLED nero	OLED_ENABLED non a 1; indirizzo 0x3D invece di 0x3C; SDA/SCL invertiti

12 · Schema riepilogo pin (cheat-sheet)

```
LED matrix DIN ..... 17      Audio MCLK ... 23   Audio SDA ... 18
                                Audio BCLK ... 21   Audio SCL ... 19
Encoder SINISTRA CLK 5  DT 22 SW 15 Audio LRCLK .. 20
Encoder CENTRALE CLK 9  DT 14 SW 16 Audio TX ..... 7
Encoder DESTRA   CLK 4  DT 2  SW 3  Audio RX ..... 8
                                OLED ..... SDA 18 / SCL 19 (0x3C)
4° encoder ..... NON montato (pin 32/33 liberi)
Pulsante FILTRO .. pin 41 -> GND  (fork, lowpass per voce)
```

Alimentazioni: matrice = **5V**; encoder, OLED e audio = **3,3V**; **GND sempre in comune** tra tutto.

Buona costruzione! Quando suona, passa al [Manuale d'Uso](#).

ichosynth è un fork di **NI404** di SP_ (soundpauli) · firmware open-source MIT.